

31338 Network Servers

Lab 1b: Advanced CentOS 10 Management

Aims

1. Boot a machine into single-user mode using the boot loader.
2. Explore and modify Linux system startup procedures.
3. Understand how to change runlevels or systemd targets.
4. Familiarise system log files.

Task 1: Boot a Single-User Shell Using the Boot Loader

Start the virtual machine, or reboot it if it is already running. The virtual machine BIOS screen appears briefly before the GRUB2 boot loader starts. When the message Press any key appears, interrupt the boot process by pressing a key. The timing may take several attempts. The GRUB boot loader screen should then be displayed.

1. GRUB2 displays the kernels available for booting. Additional kernels can be installed and managed through the GRUB configuration. Highlight the required kernel and press E to edit the boot commands.
2. The screen changes to show several configuration lines. Use the arrow keys to move to the line that starts with Linux. The aim is to pass an additional option to the kernel so that it starts with a minimal Unix shell.
3. On this line, find *ro* as a separate word. Replace *ro* with the following option:

```
rw init=/sysroot/bin/sh
```
4. After confirming that the option has been entered correctly, press *Ctrl-X* to boot the machine with the modified boot parameters.

The machine will boot into a root single-user shell without requesting a password. Changing *ro*, (read-only), or *rw* (read-write), allows files to be edited because the filesystem is mounted in writable mode. In single-user mode, the actual system files are located under */sysroot*.

Single-user mode is mainly used for maintenance, especially when an administrator needs to ensure that no other users are active and that only a minimal set of processes is running. Document the GRUB2 single-user boot process in your learning journal.

This process does not require a password, so it can also be used for Linux password recovery. The associated risk is that anyone with physical access to the machine may be able to use the same method. GRUB2 can be secured with a password, and rescue mode can also provide a single-user environment while still requiring the root password.

Task 2: Explore and Modify System Startup Scripts

From single-user mode, return to graphical mode. The simplest approach in this lab is to use the reboot command.

Linux can boot into different operating modes. Single-user mode is one example. Multi-user mode without graphics is another, and graphical mode is a third. These were previously called runlevels. In current systemd-based releases, they are called targets.

Use the following command to check the current default target:

```
systemctl get-default
```

The output should be graphical.target, which indicates that the system normally starts in graphical mode. The default target can be changed with systemctl set-default.

The main targets include:

- emergency.target - used for emergency system recovery, similar to a single-user shell.
- rescue.target - rescue mode for system recovery; it requires the root password.
- multi-user.target - multi-user mode without a graphical login.
- graphical.target - full graphical mode.

Change to multi-user mode with:

```
systemctl isolate multi-user.target
```

Document what happens in your learning journal. Then return to graphical mode with:

```
systemctl isolate graphical.target
```

Finally, examine emergency.target and rescue.target. Record the behaviour of each target in your learning journal.

The systemctl command manages the startup process, including the services that start at boot time. After exploring targets, examine services with:

```
systemctl list-unit-files
```

This command lists systemd units. Units can include targets, services, and other types. In this lab, focus on targets and services. For services, the output shows whether each service is enabled or disabled.

Check whether the sshd service is enabled or disabled by default. Also check the httpd service.

A single service can be checked with the is-enabled command, for example:

```
systemctl is-enabled sshd
```

Enabled means that a service starts automatically at boot. Disabled means that it does not. A service can also be active, meaning currently running, or inactive, meaning not running.

```
systemctl is-active sshd
```

The following commands start, stop, enable, or disable a service:

```
systemctl start <servicename>
systemctl stop <servicename>
systemctl enable <servicename>
systemctl disable <servicename>
```

Verify each result with `is-enabled` and `is-active` and record the observations in your journal.

If you like to enable and start a service in one command, you can try:

```
systemctl enable --now <servicename>
```

Use additional documentation or web resources to learn more about how `systemctl` manages the startup process.

Task 3: Examine System Log Information

Run the `dmesg` command to view the kernel ring buffer. What kinds of information are displayed? Compare this with the output of:

```
journalctl --dmesg
```

Examine the file `/var/log/messages`. What kinds of messages are stored there? Then inspect `/var/log/secure` and compare its contents.

Hint: if a log file cannot be read, check its permissions with:

```
ls -l
```

You may need to become root with `su` or adjust permissions with `chmod`, depending on the task and the file.

Experiment with `journalctl` using the following examples. Document the output in your learning journal, and look up any options that are unclear.

```
journalctl --since "10 minutes ago"
journalctl --since "2020-01-01" --until "yesterday"
journalctl -u multi-user.target
journalctl -u sshd.service
journalctl -u sshd.service --since "10 minutes ago"
journalctl -p err
journalctl -p warning
journalctl -p err --since "1 hour ago"
```

Remember to record new observations in your learning journal.